

# Sistema de gestión del V Congreso Argentino de Agroecología

---

Trabajo Final Integrador — **Grupo 1**

De la maqueta al sistema desplegado: seis entregas, una aplicación en producción.

**Integrantes:** Benjamín Rodríguez Mantilla · Alcides Castro · Lucas Budnik

Jakarta EE 11

Jersey 3.1

Hibernate 7 · MySQL

Angular 19

Docker + GitLab CI

# 01

## El problema

Qué congreso hay que gestionar, cómo nos acercamos y para quién construimos.

# Un congreso no es un formulario: son cinco procesos en paralelo

2 min

El **V Congreso Argentino de Agroecología** (FCAyF–UNLP) reúne durante varios días a cientos de personas: docentes, productores, estudiantes y organizaciones. Hoy eso se coordina con planillas, mails y formularios sueltos.

## Inscribirse y pagar

Categorías con aranceles distintos, transferencia o efectivo, comprobantes, recibos y facturas.

## Enviar y evaluar trabajos

Resúmenes por eje temático, revisión por pares a ciegas, dictámenes, desempates y devoluciones.

## Armar el programa

Mesas, pósters, talleres y conferencias en aulas con cupo y franjas horarias que no pueden solaparse.

## Comunicar

Circulares, avisos por etapa y recordatorios: hoy, listas de correo copiadas a mano.

## Certificar

Asistencia, presentación y evaluación. Al cierre, cada persona debe poder descargar lo suyo.

## El costo de no tenerlo

Datos duplicados, plazos que se pasan, trabajos sin evaluador y certificados armados uno por uno.

# Cómo nos acercamos al problema

- **Partimos del congreso real.** Del primer encuentro sobre el V Congreso Argentino de Agroecología 2027 y de lo documentado en ediciones anteriores (crecimiento de asistentes y trabajos, problemas con mails y planillas), para entender qué necesitaba realmente el sistema.
- **Primero un evento fijo; después plazos del comité.** Modelamos un congreso de tres días. Ahí vimos que casi todo lo que el sistema permite o bloquea depende del momento: inscripción, envío, evaluación, certificados. Pensando en otras ediciones, esos plazos los hicimos configurables: que dependan de la gestión del comité organizador, no de fechas fijas en el código.
- **De la organización real a roles, épicas e historias.** Vimos que el congreso necesitaba mejor organización porque hoy distintas personas hacen **tareas a mano** (inscribir y cobrar, enviar y evaluar trabajos, armar el programa, comunicar). Eso definió **roles concretos**, no un usuario genérico. Con esa base armamos la solución por **épicas** —es decir, grandes módulos del sistema: usuarios, inscripción, trabajos, cronograma, comunicación— y, dentro de cada una, **historias de usuario** (qué quiere hacer cada rol y para qué). Antes de programar las probamos en un prototipo navegable, solo de interfaz.

# Usuarios y adoptantes de la solución

6 roles

Salieron de las tareas que hoy se hacen a mano: de ahí definimos roles y armamos la solución por épicas (módulos) e historias de usuario.

| ROL                                          | QUÉ HACE EN EL SISTEMA                                                                                                                                                                                                                                                                                                                                                                  | POR QUÉ LE IMPORTA                                                                     |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Asistente                                    | Se crea una cuenta e inicia sesión · se inscribe al congreso y completa sus datos · paga la inscripción (transferencia o efectivo) y sube el comprobante · consulta si su pago fue aprobado · arma su agenda personal del congreso sin chocar horarios · puede proponer un taller · descarga su certificado cuando el congreso termina.                                                 | Sabe en todo momento si su inscripción está bien, sin mandar mails para preguntar.     |
| Autor                                        | Además de lo del asistente: carga un trabajo (artículo o relato) con coautores y eje temático · sube el PDF · lo guarda como borrador o lo envía a evaluación · ve en qué estado está y lee la devolución · cuando lo aceptan, ve cuándo y dónde presenta.                                                                                                                              | Sigue su trabajo de punta a punta y recibe la devolución sin intermediarios.           |
| Evaluador                                    | Recibe trabajos de su eje temático · acepta o rechaza la asignación · lee el PDF · da su dictamen (aceptar, rechazar u observaciones) · puede adjuntar una corrección · no evalúa trabajos propios.                                                                                                                                                                                     | Solo recibe trabajos de su especialidad, con un cupo claro y sin conflicto de interés. |
| Comité académico<br>(organizador científico) | Hace la <b>prevalidación de los trabajos</b> (revisión inicial antes de asignar evaluadores) · asigna evaluadores por eje temático respetando cupos · resuelve empates y reenvíos · aprueba solicitudes de nuevos evaluadores · configura ejes, modalidades y plazos de envío · consulta estadísticas del proceso académico.                                                            | Es el <b>adoptante principal</b> : hoy hace todo esto a mano.                          |
| Administración                               | Da de alta y gestiona usuarios · define <b>aranceles</b> (cuánto paga cada categoría de inscripción) · valida pagos, emite recibos/facturas y hace el <b>arqueo de caja</b> (control de lo cobrado en el día) · carga aulas, horarios y el programa del congreso · publica circulares y avisos · cierra el congreso y habilita certificados · ve reportes y pendientes de organización. | Centraliza la operación del evento y el control del dinero.                            |

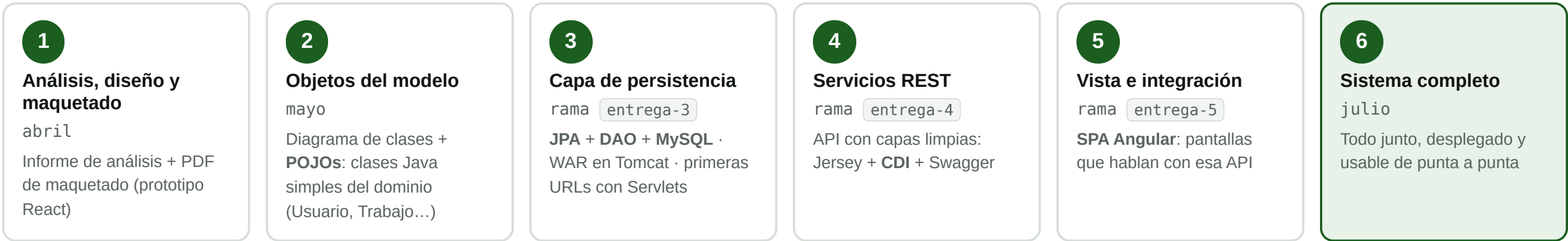
# 02

## Las seis entregas

Qué objetivo tuvo cada etapa, qué conceptos de la cátedra aplicamos, con qué nos peleamos y qué quedó funcionando.

# El camino: de la maqueta al sistema desplegado

abril → julio 2026



## Una regla que nos salvó

Cada entrega vivió en su propia rama ( entrega-3 , entrega-4 , entrega-5 ) y se integró a main sólo con el pipeline en verde. Nunca perdimos una entrega ya aprobada por trabajar sobre la siguiente.



# Análisis, diseño y maquetado

Producto: informe + PDF de maquetado

## Objetivo

Entender el dominio del V Congreso y acordar *qué* construir, con pantallas discutibles. Relevamiento → roles → **iniciativa** → **épicas** → **HU** → prototipo cliente–servidor (aún sin CDI).

### ① Análisis — HU más importantes para el congreso

- **HU-01 / HU-02** — Registrarse e iniciar sesión (entrada al sistema).
- **HU-06** — Solicitar inscripción y adjuntar comprobante de pago.
- **HU-07** — Admin valida pagos (aprobar/rechazar) y habilita al asistente.
- **HU-11** — Enviar un trabajo (PDF, eje, oral/póster) a evaluación.
- **HU-12** — Evaluador dictamina (aprobar/rechazar + devolución opcional).
- **HU-13 / HU-14** — Admin crea actividades y publica el programa.
- **HU-17** — Asistente arma su agenda personal sin solapar horarios.

Salen del informe *Análisis del Problema Sobre el V Congreso* (épicas 1–5).

## Dificultad

Al maquetar aparecieron reglas que no estaban escritas: ¿qué pasa si dos evaluadores empatan?, ¿puede un evaluador revisar su propio trabajo?

**Cómo la resolvimos:** las anotamos en el análisis / HU y las llevamos al modelo de la Entrega 2.

② Maquetado: `archivo_maquetado.pdf` — capturas del prototipo **React + Vite** (`localStorage`, sin backend). Siguiente diapositiva: pantallas ligadas a esas HU.



# Del registro al programa: capturas del prototipo

archivo\_maquetado.pdf

22/4/26, 13:13

Untitled

V Congreso Argentino de Agroecología  
La Plata 2027

Iniciar sesión

Registrarse

Registrarse

Crea tu cuenta para participar

Nombre

Apellido

Email

Contraseña

Confirmar contraseña

Registrarse

¿Ya tienes cuenta? Iniciar sesión

HU-01 Registro

V Congreso Argentino de Agroecología  
La Plata 2027

julieta

Inscripción al Congreso

Completá tus datos y adjuntá el comprobante de pago

Institución

Provincia

Selecciona

Categoría de Inscripción

Selecciona una categoría

Comprobante de Pago

Seleccionar archivo Sin archivos seleccionados

HU-06 Inscripción + comprobante

V Congreso Argentino de Agroecología  
La Plata 2027

Admin

Panel de Administración

Gestión completa del congreso

Crear Mesa Temática

Crear Mesa Redonda

Crear Sesión de Pósters

Crear Taller

Crear Conferencia

Programa publicado

Controla si el cronograma es visible para el público en "Programa".

Publicado

Validación de Inscripciones

julieta lopez

Ver comprobante

Aprobar

Rechazar

HU-07 Admin — Aprobar / Rechazar

# Modelo de objetos → por qué DAOs

separar “el negocio” de “la base”

Diseñamos el congreso como **objetos Java del negocio** (POJOs): cada cosa importante es una clase con datos, estado y relaciones. Ese diseño **no guarda nada en MySQL todavía** — solo dice *qué* existe. Ejemplo: `Trabajo` (titulo, PDF, `enviar()`), `AsignacionEvaluacion`, `Evaluacion` (y lo mismo para `Usuario`, `Pago`, `InscripcionCongreso` ...). Cuando el servidor se apaga, esos objetos en memoria **se pierden**: hace falta **persistencia**. **Pregunta de diseño**: ¿quién guarda y busca esos objetos en MySQL (un `Trabajo`, un `Usuario`, un `Pago` ...)? Si lo metemos en la clase del negocio o en el servlet, mezclamos negocio con MySQL. Por eso elegimos el patrón **DAO**: una responsabilidad aparte.

El modelo ya estaba partido en entidades claras. Eso empujó a un camino simple: **una entidad del diagrama** → **una tabla** → **un DAO**. ¿Por qué no meter MySQL adentro de las clases?

### Camino que evitamos

La alternativa era escribir el código que habla con MySQL ( `EntityManager`, consultas) dentro de `Usuario` o del servlet HTTP. Problemas:

- `Usuario` pasaría a depender de cómo está armada la base: si cambia MySQL, hay que tocar la clase de reglas del usuario, aunque esas reglas no tengan nada que ver con la base.
- Si diez pantallas necesitan guardar o buscar un usuario, la misma lógica de `persist` / `find` se copia diez veces, en vez de estar en un solo lugar.
- Se vuelve imposible probar las reglas del negocio sin tener la base levantada, porque quedan mezcladas en el mismo código.

En resumen: mezcla dos responsabilidades que deberían estar separadas — las reglas del congreso y la comunicación con MySQL.

### Camino que elegimos: DAO

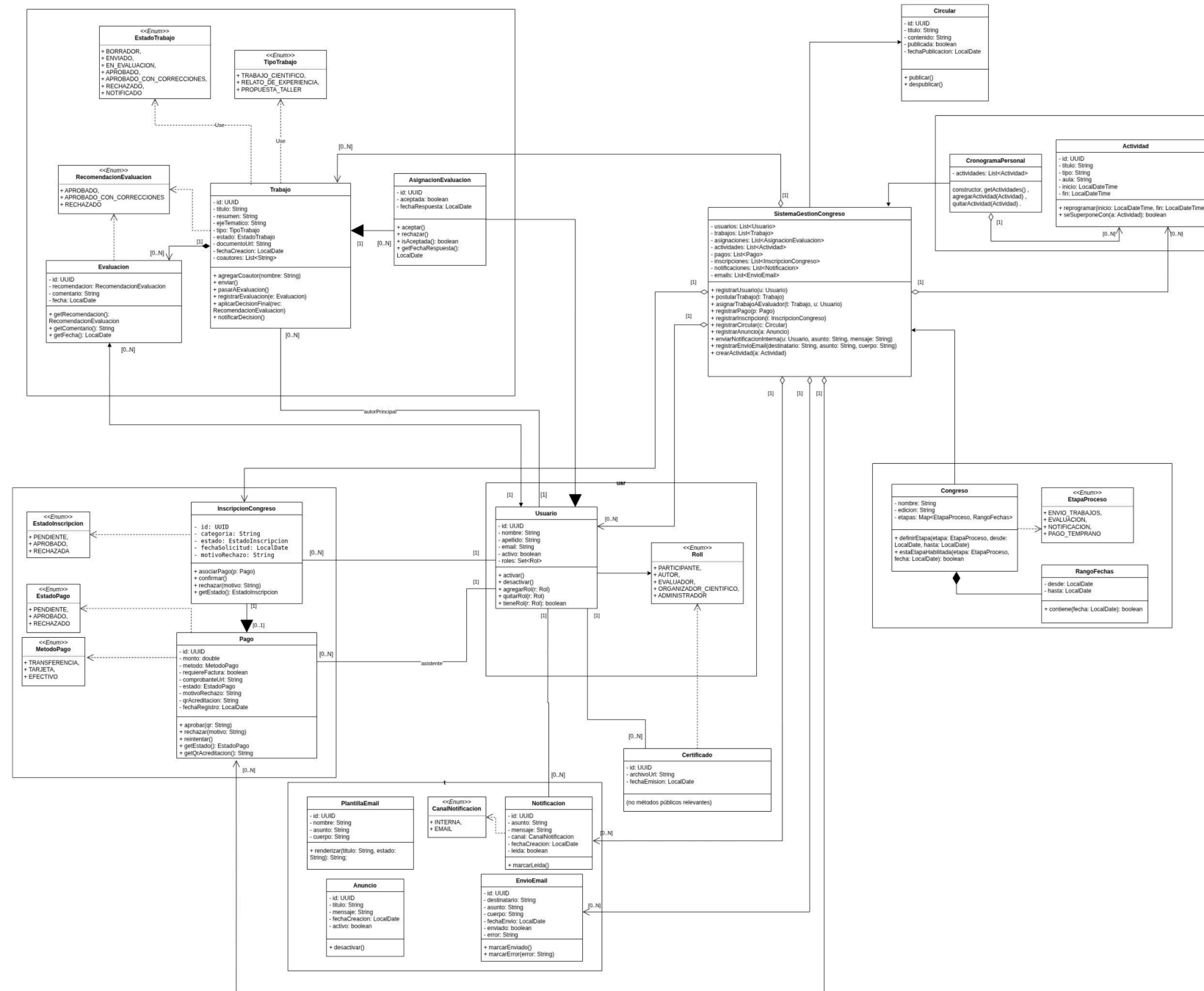
**DAO** = Data Access Object: es un **patrón de diseño** (no “magia de JPA”). En el código es una interfaz + implementación cuya **única** responsabilidad es el acceso a datos de **una** entidad: guardar / buscar / actualizar / borrar.

- `Usuario` = datos y reglas del usuario (modelo E2).
- `UsuarioDAO` + `UsuarioDAOImpl` = cómo entra y sale de MySQL (E3).
- **JPA** ( `@Entity`, `EntityManager` ) = la herramienta que usa el DAO para mapear objeto ↔ fila.

**Por eso el modelo nos llevó a DAOs**: ya teníamos objetos bien cortados; el DAO es el “portero” de cada uno hacia la base, sin ensuciar el diagrama de negocio. En la E3 el Servlet todavía mezcla puerta HTTP y caso de uso; en la E4 terminamos el corte: `Resource` → `Service` → `DAO`.

### Camino del diseño

HU → clase del diagrama → POJO → entidad JPA → tabla MySQL → DAO (CRUD de esa entidad) → Factory (un solo lugar crea el DAO) → en E4, CDI inyecta esas dependencias.



# Del diagrama a la persistencia (idea fija)

una caja → un DAO

Cuando hagan zoom en el diagrama, piensen esto: **cada clase con `id` es candidata a tabla y a DAO.**

| EN EL MODELO (E2)                | EN LA PERSISTENCIA (E3)    | QUIÉN LO HACE                                         |
|----------------------------------|----------------------------|-------------------------------------------------------|
| Usuario                          | tabla <code>usuario</code> | <code>UsuarioDAO</code>                               |
| Trabajo                          | tabla <code>trabajo</code> | <code>TrabajoDAO</code>                               |
| Pago                             | tabla <code>pago</code>    | <code>PagoDAO</code>                                  |
| AsignacionEvaluacion             | su tabla                   | <code>AsignacionEvaluacionDAO</code>                  |
| Evaluacion                       | su tabla                   | <code>EvaluacionDAO</code>                            |
| CRUD repetido (save/find/delete) | código compartido          | <code>GenericDAO</code> / <code>AbstractJpaDAO</code> |
| “¿quién me da el DAO?”           | un solo lugar de creación  | <code>DAOFactory</code> (Factory)                     |

Nuestro diseño de objetos partió el dominio en entidades. Eso hizo natural el patrón **DAO**: cada entidad se persiste por su cuenta, sin meter MySQL dentro de las clases del negocio. **JPA** convierte el objeto en una fila de la tabla (y al revés). El **DAO** es quien pide ese guardar/buscar. El **Factory** es el único lugar que crea los DAOs: el resto del sistema no tiene que hacer `new UsuarioDAOImpl()` en cada pantalla.

# Persistir el modelo · qué aprendimos a usar

rama entrega-3

Venimos de la E2: ya teníamos el **diseño del dominio**. El objetivo de la E3 fue **persistirlo** en MySQL (capa JPA, patrón DAO con ABM, WAR en Tomcat). Antes los datos vivían en `localStorage`; acá pasan al servidor. Todavía no es la API limpia con CDI (E4).

Para lograr esto necesitamos una **app web en Java**: un servidor que ejecute la aplicación, herramientas que conecten el modelo con la base, y un patrón que ordene el acceso a los datos. Empecemos por la base: **cómo se ejecuta la aplicación** (WAR + Tomcat).

## 1 · WAR + Tomcat 10 (Jakarta)

La aplicación se **empaqueta en un solo archivo desplegable** (WAR). Un servidor de aplicaciones —Tomcat— es el encargado de ejecutarlo. Usamos la especificación moderna de Java para la web: **Jakarta EE** (Tomcat 10), que define cómo se escriben y despliegan este tipo de sistemas.

## 2 · Ciclo de vida (ServletContextListener)

Una app web no “está siempre prendida” de la misma forma: tiene un **ciclo de vida**. Al **arrancar**, se conecta a la base de datos y, si hace falta, carga datos iniciales para poder probar. Al **apagar**, se cierra todo de forma ordenada para no dejar conexiones ni recursos abiertos.

## 3 · Servlet

Cada funcionalidad importante tiene un **responsable** que atiende los pedidos que llegan por una dirección concreta (URL) y devuelve una respuesta según lo pedido: listar, crear, probar la persistencia, etc.

## 4 · Cliente–servidor + JSON

El **cliente** (navegador u otra herramienta) pide algo; el **servidor** responde. Esa respuesta puede ser **datos estructurados (JSON)**, pensados para que otro programa los use, o una **página** pensada para que la lea una persona (por ejemplo, el resultado de las pruebas de ABM).

## 5 · Persistencia orientada a objetos

En lugar de escribir a mano instrucciones de base de datos en cada pantalla, trabajamos con **objetos del dominio** (usuario, trabajo, pago...). Existe una **capa intermedia** que traduce esos objetos a datos guardados en tablas —y al revés al leerlos.

**En una frase:** empaquetamos y desplegamos la app → al arrancar queda lista la base → cada URL tiene quién la atiende → cliente y servidor se hablan → los objetos del modelo se persisten sin mezclar eso en cada pantalla.

# Cómo las usamos, de punta a punta

no solo “cuál” · también “cómo”

No es una lista suelta: es la cadena que lleva lo que programamos hasta la app que corre en el servidor del curso.

## Paso A · Armar el producto (Maven)

El código no se puede ejecutar tal cual en un servidor: hay que **empaquetarlo** en un solo archivo (WAR) con todo lo necesario. **Maven** arma ese paquete automáticamente: revisa qué librerías necesita la app (Hibernate, MySQL, Gson, Logback...), las descarga, compila y genera el WAR (`mvn package` → `jyaa2026-grupo1.war`).

## Paso B · Hablar con la base (Hibernate + MySQL)

La app piensa en **objetos** (usuario, trabajo...); la base guarda **tablas**. **MySQL** es el motor que persiste esos datos en disco (`jyaa2026_bd1`). **Hibernate (JPA)** es el traductor automático entre ambos mundos: nosotros trabajamos con objetos; él genera el SQL. La configuración vive en `persistence.xml`.

## Paso C · Responder al cliente (Gson + Logback)

El navegador no entiende objetos Java: hace falta un formato universal. **Gson** los traduce a **JSON** para que el cliente pueda usarlos. **Logback** es el “cuaderno de bitácora”: registra qué pasó (incluido el SQL) para detectar errores (`logback.xml`, consola de Tomcat).

## Paso D · Llevarlo al servidor (Docker + GitLab CI)

En el servidor del curso no se instala cada proyecto a mano: se arma una **caja** (contenedor Docker) con Tomcat y el WAR listos para correr. **GitLab CI** automatiza ese proceso en un **pipeline**: cada vez que subimos código, se construye la imagen, se reemplaza el contenedor y **Traefik** publica la URL del grupo. Referencia: `Dockerfile` · dominio del curso.

# Cómo el diagrama de la E2 se vuelve tablas

objeto Java ↔ fila MySQL

En la E2 el modelo vivía en memoria. En la E3 Hibernate se encarga de **guardarlo y recuperarlo** en MySQL, sin que escribamos el SQL a mano en cada pantalla.

## ¿Qué es una entidad?

En la E2 diseñamos clases Java que representaban cosas del congreso (`Trabajo`, `Usuario`), pero eran solo objetos en memoria. Una **entidad** es esa misma clase, marcada con `@Entity`, para decirle a Hibernate que debe guardarse en una tabla. El `@Id` identifica cada fila de forma única (como un DNI). Por ejemplo, `Trabajo` se convierte en la tabla de trabajos, y la relación con el autor (`@ManyToOne`) en una columna con el id de ese usuario (`autor_id`).

## Relaciones = flechas del diagrama

El diagrama de la E2 mostraba relaciones (un trabajo tiene muchas evaluaciones, una inscripción un solo pago, etc.). En la base eso son **columnas que referencian otra fila**. Hibernate necesita el tipo de cada relación: `@OneToMany` / `@ManyToOne` (un trabajo, muchas evaluaciones; cada evaluación, un solo trabajo); `@OneToOne` (una inscripción, un único pago); `@ManyToMany` (actividad con varios trabajos y viceversa); `@ElementCollection` (listas simples: roles, coautores); `@Enumerated` (estados fijos como `EstadoTrabajo`).

## `persistence.xml`

Es el archivo donde le decimos a Hibernate **cómo y dónde** conectarse: qué base, con qué usuario, y qué clases tratar como entidades. En Tomcat hay que **listar a mano** las ~15 clases: no las descubre solo.

## EMF y EntityManager

El **EntityManagerFactory (EMF)** es la conexión general a la base: se crea **una sola vez** al arrancar (lo hace `JpaUtil`) y permanece abierta mientras corre el servidor.

El **EntityManager (EM)** ejecuta cada operación (guardar, buscar, actualizar, borrar): se abre para esa operación y se cierra apenas termina.

La **transacción**: si todo sale bien, `commit`; si falla a mitad, `rollback` (nada a medias).

Quién abre el EntityManager cada vez: el **DAO**. Quién lo crea: la **Factory**.



# Quién guarda en MySQL (y por qué no el Servlet)

Si el Servlet hiciera SQL, mezclaríamos “atender pedidos HTTP” con “hablar con la base”. Por eso cada entidad tiene su DAO, y el Factory es el único lugar que crea esos DAOs.

## DAO = Data Access Object (patrón)

El DAO es el “portero” de una sola entidad: el único responsable de guardar, buscar, actualizar o borrar sus datos (por ejemplo, todo lo de un trabajo pasa por `TrabajoDAO`). No es magia de JPA, sino una **decisión de diseño**: en código es una interfaz con su implementación (`TrabajoDAO` / `TrabajoDAOImpl`).

## Cómo lo apilamos

Todas las entidades necesitan las mismas operaciones básicas (guardar, buscar, actualizar, borrar). En vez de repetir esa lógica, se define una sola vez: `GenericDAO<T>` declara el ABM común, y `AbstractJpaDAO` lo implementa (abre el EntityManager, arranca la transacción, ejecuta `persist` / `merge` / `remove` / `find`, y cierra). Cada entidad hereda eso y solo agrega consultas propias (por ejemplo, buscar por email).

## Factory

Si cada Servlet o test creara el DAO con `new TrabajoDAOImpl()`, esa línea se repetiría en muchos archivos y un cambio de implementación obligaría a tocarlos todos.

`DAOFactory.getTrabajoDAO()` centraliza la creación: es un **patrón creacional**; todos piden el DAO ahí y no les importa cómo está armado por dentro.

## Ejemplo completo: guardar un trabajo

1. Llega un pedido `POST /api/trabajos`: el Servlet lee el JSON del cliente.
2. Pide el DAO al Factory con `DAOFactory.getTrabajoDAO()` (y el de usuario si hace falta).
3. Arma el objeto `Trabajo` en memoria: todavía no está en la base.
4. Llama a `trabajoDAO.alta(t)`: ahí se abre la transacción, se guarda el objeto (`persist`) y se confirma el cambio (`commit`).
5. Hibernate genera el `INSERT` y MySQL asigna el `id`.
6. Gson arma la respuesta 201 con el trabajo ya persistido.

**Prueba de la entrega:** `/test-persistencia` ejecuta ~14 ABM (usuario, inscripción y pago, trabajo, asignación y evaluación...) y muestra OK o FALLO. Así respondemos la pregunta: **quién guarda y busca esos objetos en MySQL** es el DAO, no el Servlet ni la clase del negocio. Con eso la E3 demuestra que el modelo ya vive en MySQL; en la siguiente vemos qué nos costó y qué quedó para la E4.



# Qué nos costó · resultado de /test-persistencia

13 OK · 1 FALLO (lazy)

## ① “Unknown entity” en Tomcat

Hibernate se queja porque no reconoce una clase como entidad, aunque tenga `@Entity`. A diferencia de otros entornos, Tomcat no escanea el proyecto para descubrirlas solo. Lo resolvimos declarando cada clase a mano con una etiqueta `<class>` dentro de `persistence.xml`.

## ② Lazy loading al listar

Por defecto, algunas relaciones no se traen de la base hasta que se usan. Si se accede a esa colección **después** de cerrar la sesión, Hibernate falla (*could not initialize proxy — no Session*). `/test-persistencia` lo dejó en evidencia: ~13 ABM en OK y un FALLO en `Actividad.trabajos` (salida a la derecha). En el resto lo corregimos con `JOIN FETCH` en los DAOs; este caso pendiente muestra que la batería de pruebas sirve para detectar huecos reales.

## ③ Capas mezcladas (límite de la E3)

El Servlet pedía el DAO al Factory y manejaba el acceso a datos él mismo: mezclaba la puerta HTTP con el caso de uso y el acceso a MySQL. Alcanzó para demostrar el ABM, pero no es una API prolija. En la E4 separamos en tres capas — `Resource` (puerta HTTP), `Service` (aplicación / caso de uso) y `DAO` (infraestructura / MySQL) — dejando el dominio (`Usuario`, `Trabajo` ...) sin HTTP ni SQL. **CDI** (`@Inject`) las conecta y **Jersey** atiende HTTP.

**Cierre:** en la E3 el modelo **se persiste** y se despliega; la prueba ABM lo verifica en vivo. En la E4 ordenamos capas con **DIP**: la web ya no crea DAOs a mano — el contenedor los inyecta.

## Pruebas ABM – DAO + JPA

### HAY TESTS FALLIDOS

[Congreso] OK

[Usuario] OK

[Pago] OK

[Inscripcion + Pago] OK

[Trabajo] OK

[Actividad + M2M trabajos] FALLO  
lazy init `Actividad.trabajos` – no Session

[Circular ... Certificado ...  
Asignacion+Evaluacion] OK

--- Resumen --- OK: 13 | Fallos: 1

# De la Práctica 6 al problema que resolvimos

rama entrega-4

## Lo que ya sabíamos (Práctica 6) — el recorrido

### 1 · Del cliente a Jersey

El cliente manda un `POST` con JSON a `/api/usuarios`. Tomcat recibe el pedido y, como la URL empieza con `/api/*`, se lo pasa al `ServletContainer` de Jersey (registrado en `web.xml`). Jersey todavía no persiste nada: solo enruta según `@Path("/usuarios")`, `@POST`, `@Consumes` y `@Produces`.

### 2 · Jackson arma el objeto y se ejecuta `alta`

El método pide un parámetro `Usuario`, no un texto. Jersey busca un `MessageBodyReader`; encuentra el de **Jackson**, que parsea el JSON y construye el objeto. Recién ahí se ejecuta `UsuarioResource.alta(...)` con el usuario ya armado.

### 3 · El Resource le pasa el objeto al DAO

Desde `UsuarioResource` se recibe el objeto parseado y se lo entrega al DAO. El Resource no sabe SQL, no arma el JSON a mano y no abre la conexión a MySQL: solo atiende HTTP y delega la persistencia.

### 4 · Hibernate traduce el objeto a un `INSERT`

Al arrancar, `JpaBootstrapListener` crea una sola vez la `EntityManagerFactory`. En cada operación el DAO obtiene un `EntityManager` y llama a `em.persist(usuario)`. Hibernate lee `@Entity` y `@Table` de la clase y arma el `INSERT` que ejecuta MySQL.

### 5 · La respuesta vuelve en JSON

El método hace `return` del resultado. Jersey lee `@Produces`, busca un `MessageBodyWriter` y Jackson convierte el objeto otra vez a JSON. El cliente recibe HTTP 200 (o el código que corresponda) con ese body. La Entrega 4 no tira abajo este recorrido: lo extiende.

### El problema

En la Práctica 6, `UsuarioResource` dependía de la clase concreta `UsuarioDAOImpl`. Si cambiábamos cómo se persiste, había que tocar la capa HTTP. Esa capa no debería saber quién fabrica el DAO ni cómo habla con MySQL.

**Qué pedía la E4:** separar en capas, que el Resource deje de conocer `UsuarioDAOImpl`, y que **CDI / Weld** conecte las piezas con `@Inject`.

# Cuatro capas, una responsabilidad cada una

presentación · servicio · dominio · infraestructura

## Presentación

Recibe el pedido HTTP y arma la respuesta (201, 400, JSON). Ejemplo: `UsuarioResource`. No sabe SQL ni cómo se abre la conexión a la base.

## Dominio

Qué es cada cosa del negocio: `Usuario`, `Trabajo`, estados y roles. Sin HTTP y sin detalles de MySQL.

## Aplicación / Servicio

Coordina el caso de uso: valida reglas y recién ahí pide guardar o buscar. Ejemplo: `UsuarioService` (email único, rol por defecto).

## Infraestructura

Habla con el mundo técnico: la implementación del DAO, Hibernate, MySQL, y el almacenamiento de archivos PDF en disco.

### Qué sí y qué no puede saber `UsuarioResource`

La razón de fondo: cada capa solo conoce a la de al lado, nunca a las de más abajo. El Resource habla con el Service; el Service habla con la interfaz del DAO; la implementación del DAO habla con MySQL. Si el Resource conociera `UsuarioDAOImpl`, un cambio en cómo se guarda obligaría a tocar la puerta HTTP. Por eso el Resource no “salta” capas.

**Sí:** que existe `UsuarioService`, que puede pedirle el alta de un usuario, códigos HTTP, y que el cuerpo del pedido es un usuario o un DTO.

**No:** que la implementación se llama `UsuarioDAOImpl`, nombres de tablas, ni cómo se abre o cierra la sesión con la base.

### DTOs: entrada y salida

Un **DTO** (Data Transfer Object) es un objeto pensado solo para viajar por la red: decide exactamente qué datos entran o salen en el mensaje. Si devolvemos la entidad completa del usuario, mandamos de más: la contraseña, y relaciones entre objetos que pueden hacer que la respuesta se enrede en un ciclo infinito. El DTO corta ese problema: controlamos qué viaja en el mensaje.

En el proyecto hay dos casos concretos de esa idea. **Entrada:** en el login solo hacen falta el email y la contraseña; no hace falta mandar todo el usuario. **Salida:** cuando respondemos con los datos de un usuario, el mensaje no incluye la contraseña.

A futuro, separar el DTO de la entidad protege el contrato con quien consume la API (Swagger, el frontend, otro cliente). Si el modelo de usuario cambia o crece en próximas entregas, ese cambio queda contenido en el dominio: no se filtra solo al JSON que ya reciben los clientes. Quien decide qué se muestra es el DTO, no la entidad.

# Interfaz en el medio · alguien te pasa lo que necesitás

DIP · DI · tres formas

## DIP — la pregunta

`UsuarioResource` no puede conocer `UsuarioDAOImpl`. Entonces el Service, ¿a qué le habla? A la interfaz `UsuarioDAO`: contrato de *qué* se puede pedir, sin decir *cómo*. Eso es el **Principio de Inversión de Dependencias**: alto nivel no depende del bajo nivel; ambos miran la abstracción. Antes (Práctica 6) el Resource conocía el Impl; ahora el Service depende de la interfaz.

## DI — cómo se cumple DIP

En lugar de que el Resource cree el DAO a mano, alguien desde afuera entrega la dependencia ya lista.

### Tres formas de inyectar

#### a) Constructor

Se pasan al crear el objeto. Más recomendada en general; facilita tests.

#### b) Campo (field)

Campo + contenedor completa. La más corta con CDI y Jersey; la que usamos.

#### c) Setter

Método de asignación. Útil si la dependencia es opcional.

## Cómo Weld elige qué entregar

Con `@Inject`, Weld busca un bean compatible con el tipo pedido. Si pedís una clase concreta (por ejemplo el Service), entrega el bean de esa clase. Si pedís una interfaz (por ejemplo el DAO de usuario), busca cuál bean la implementa; si hay una sola, la entrega automáticamente. Eso conecta los dos lados de la idea DIP: DIP pone la interfaz en el medio; Weld es **quién conecta** el lado que pide el contrato con el lado que lo implementa, sin que el alto nivel conozca la clase concreta.

**Por qué elegimos field injection en esta entrega:** con CDI y Jersey es la forma más corta de escribirlo, y es lo que pedía el práctico. La inyección por constructor sería más testeable; field injection es lo que aplicamos acá.

# Ya no manejamos el ciclo de vida · estándar y motor

quién crea · inyecta · destruye

## Antes (control del programador)

El programador decidía cuándo y cómo nacería cada objeto (o lo centralizaba a mano en un Factory).

## Ahora (IoC)

**Weld** decide cuándo crear, entregar y destruir. El programador solo declara qué necesita con `@Inject`. Por eso se llama “inversión”: el control pasó al framework.

## La pregunta que queda de IoC

Dijimos que “el framework” decide crear, inyectar y destruir. Eso no es magia abstracta: hace falta un **estándar de reglas** (qué anotaciones existen y qué significan) y una **herramienta concreta** que las ejecute al arrancar y en cada petición.

## CDI = el estándar

Contrato de Jakarta: `@Inject`, scopes, `@Produces`, `@Disposes`. Solo dice las reglas; por sí solo no crea nada.

## Weld = quien las cumple

Lee las anotaciones, crea, inyecta y destruye. Sin Weld, las anotaciones no tienen efecto.

## Cómo se entera Weld de qué clases administrar

Al arrancar, Weld recorre las clases del proyecto buscando cuáles tienen anotación de scope (`@RequestScoped` o `@ApplicationScoped`). A esas las registra como **bean**: objetos cuyo ciclo de vida gestiona él. Eso se habilita con `beans.xml` en modo `bean-discovery-mode="annotated"`: “escaneá solo lo anotado”. Sin ese archivo, Weld no arranca y `@Inject` no funciona.

## Por qué el Resource también tiene que ser bean

Jersey atiende HTTP, pero no inventa las dependencias solo. Para poder inyectarle el Service al Resource, esa clase tiene que pasar por Weld. El puente entre Jersey y CDI es `jersey-cdi1x-servlet`. Si el Resource no fuera bean, Jersey lo crearía a mano y lo que debería inyectarse quedaría vacío.

**Siguiente:** cuánto dura cada bean (por petición o por toda la app) y cómo una fábrica permanente fabrica un EntityManager distinto por pedido.

# Cuánto dura cada bean (y la fábrica vs lo fabricado)

## @RequestScoped — una instancia por petición

En un `POST /api/usuarios`, Weld crea para ese pedido un `UsuarioResource`, un `UsuarioService`, un `UsuarioDAOImpl` y el `EntityManager` de esa petición. En un pago, crea `PagoResource`, `PagoService` y los tres DAOs que coordina (`PagoDAO`, `InscripcionCongresoDAO`, `UsuarioDAO`), todos compartiendo el **mismo** `EntityManager` de esa petición puntual. Al terminar el pedido, **todas esas instancias se destruyen juntas**: no queda estado colgado para el próximo cliente.

## @ApplicationScoped — una sola instancia para toda la app

Ejemplos: `EntityManagerProducer` y `DocumentStorageService`. Esos dos no necesitan renovarse en cada pedido porque **no guardan datos de una petición particular**: el `Producer` solo sabe cómo crear y cerrar `EntityManagers`; el servicio de documentos solo sabe guardar y leer PDFs en disco. Una instancia permanente alcanza para toda la vida de la aplicación.

## ¿No es contradictorio? Fábrica permanente, producto por petición

`EntityManagerProducer` es una clase única y permanente ( `@ApplicationScoped` ). Adentro tiene un método con `@Produces` que fabrica un `EntityManager` **distinto por cada petición**, y un método con `@Disposes` que lo destruye cuando esa petición termina. La fábrica es una sola; lo que fabrica es distinto cada vez. Por eso conviven `@ApplicationScoped` (la fábrica) y scope de request (el `EntityManager` producido) en el mismo mecanismo, sin contradicción.

**Puente a la siguiente:** ya se vio quién administra (Weld), qué reglas sigue (CDI) y cuánto dura cada bean ( `@RequestScoped` / `@ApplicationScoped` ). Falta ver cómo Weld decide **qué bean entregar** cuando se pide un tipo con `@Inject` (una clase concreta o una interfaz).

# Un EM por petición · fábrica una vez al levantar Tomcat

@Produces · JpaBootstrapListener

¿Por qué un solo EntityManager por petición y no uno por DAO?

## El problema — historia de un pago a medias

Imaginá que `PagoService` coordina tres DAOs y cada uno abre su propio EntityManager y su propia transacción. Primero el DAO de pago guarda el pago: todo bien, confirma y cierra. Después el DAO de inscripción guarda la inscripción: también bien, confirma y cierra. Al final el DAO de usuario falla (por ejemplo, un dato inválido). Los dos primeros **ya quedaron confirmados en MySQL** y no se pueden deshacer: hay un pago y una inscripción registrados, pero el resto de la operación no terminó. Quedó todo a medias. Lo que se perdió es la **atomicidad**: la garantía de que o se guarda **todo junto**, o **no se guarda nada**.

## La solución: EntityManagerProducer

Con `@Produces` y scope de request, Weld crea **un** EntityManager al inicio de la petición y abre la transacción. Los tres DAOs del pago ( `PagoDAO` , `InscripcionCongresoDAO` , `UsuarioDAO` ) usan ese mismo EntityManager. Al terminar, `@Disposes` cierra el ciclo: si los tres funcionaron, hace **commit de los tres juntos**; si cualquiera falló, hace **rollback de los tres juntos** — y cierra el EntityManager. Así el pago no puede quedar a medias.

**Puente:** el Producer fabrica un EntityManager por petición, pero ¿quién deja lista la fábrica antes de que llegue el primer pedido? Eso no es un gancho de bean CDI: es el arranque de Tomcat.

## ¿Por qué no @PostConstruct / @PreDestroy ?

Esos son ganchos de **CDI**, atados al ciclo de vida de un bean ( `@RequestScoped` o `@ApplicationScoped` ): corren cuando nace o muere ese bean. Acá hace falta algo que exista **antes** de que arranque cualquier bean y se cierre **después** de apagar todo el servidor. Tomcat ofrece su propio gancho: `ServletContextListener` , con `contextInitialized` al levantar y `contextDestroyed` al bajar. Eso es lo que usa `JpaBootstrapListener` , en vez del gancho de CDI.

## EMF y EM: suenan parecido, no son lo mismo

La **EntityManagerFactory** es pesada de armar: lee `persistence.xml` , prepara la conexión a MySQL, deja listo Hibernate. Se crea **una sola vez** para toda la vida del servidor: eso lo hace `JpaBootstrapListener` en `contextInitialized` , y la cierra en `contextDestroyed` . El **EntityManager** es lo que esa fábrica **produce**: uno nuevo por cada petición HTTP. Eso lo hace `EntityManagerProducer` ( `@Produces` / `@Disposes` ), usando la fábrica que el listener ya dejó lista.

**En una frase:** `JpaBootstrapListener` arma la fábrica una sola vez; `EntityManagerProducer` usa esa fábrica para sacar un producto nuevo en cada petición (y destruirlo al terminar el pedido).



POST /api/usuarios : de la red a MySQL (A–E)

cinco fases

A · Cliente y Jersey

Alguien completa el formulario de alta y lo envía. Llega `POST /api/usuarios` con JSON. Tomcat lo deriva al `ServletContainer` de Jersey (`web.xml`, `/api/*`). Jersey enruta con `@ApplicationPath`, `@Path("/usuarios")` y `@POST`. Jackson convierte el JSON en un `Usuario`. Para llamar a `alta(...)`, Jersey necesita un bean CDI, no un `new`.

B · Weld crea e inyecta

El servidor arma el equipo de este pedido. Gracias a `jersey-cdi1x-servlet`, **Weld** entrega un `UsuarioResource` `@RequestScoped`. Ve `@Inject UsuarioService` y crea el Service; ve `@Inject UsuarioDAO` y crea `UsuarioDAOImpl`. El DAO pide un `EntityManager`: Weld llama al `@Produces` de `EntityManagerProducer`, abre la transacción, y ese mismo EM lo comparten Resource, Service y DAO.

C · Resource → Service

Antes de guardar, hay que revisar si el pedido tiene sentido. `UsuarioResource.alta(...)` no llama al DAO: llama a `usuarioService.alta(usuario)` (regla de capas). El Service valida: email no repetido, minúsculas, rol `PARTICIPANTE` por defecto. Si el email ya existe, lanza `NegocioException`.

D · Aplicación, infraestructura y MySQL

Ya se validó el email; ahora hay que guardarlo de verdad. `UsuarioService` llama a `usuarioDAO.alta(usuario)`. En modo CDI, `AbstractJpaDAO` hace `persist` sobre el `EntityManager` inyectado (el mismo de esta petición; no abre otra transacción — atomicidad del Producer). Hibernate lee `@Entity` / `@Table` y arma el `INSERT`. MySQL asigna el `id`; el objeto en memoria ya lo tiene.

E · Dos salidas posibles

¿El email que mandaron ya existía o no? Según eso, el camino se bifurca.

Error de negocio

El email ya estaba: se rechaza el alta. `NegocioException` sube hasta Jersey. `NegocioExceptionMapper` (ExceptionHandler: convierte la excepción en respuesta HTTP) responde HTTP **400** con `{"error":"..."}`. `@Disposes` hace **rollback** y cierra el `EntityManager`: no queda fila a medias.

Éxito

El email era nuevo: se confirma el alta. El Resource convierte a `UsuarioDTO` (sin password), responde HTTP **201** con header `Location`, Jackson arma el JSON, y `@Disposes` hace **commit** y cierra.

Detalle de coloquio

El Resource no arma a mano el JSON de error: lanza la excepción correcta y el ExceptionMapper hace el resto. Misma idea para 404 con `NotFoundExceptionMapper`.



# Probar la API sin inventar otro camino

OpenAPI · misma petición real

## Qué es

Swagger es la interfaz visual para probar los mismos endpoints que curl o Postman. **No ejecuta lógica de negocio propia:** dispara el flujo HTTP real (Tomcat, Jersey, Weld, Service, DAO, MySQL) y muestra el código de respuesta. La página está en `/swagger-ui/` (`swagger-ui/index.html`).

## Qué hace cada anotación en la pantalla

### @Tag

Agrupa endpoints bajo un nombre visible (Usuarios, Trabajos, Pagos...).

### @Operation

Describe qué hace ese endpoint: es el título del formulario “Try it out”.

### @ApiResponse

Documenta códigos HTTP posibles (201, 400, 404...) antes de probar.

## Cómo se arma solo

`OpenApiResource` lee las anotaciones y genera `GET /api/openapi.json`. Swagger UI lee ese JSON y dibuja los formularios sola. Un endpoint nuevo bien anotado **aparece solo** en la pantalla.

## Cómo comprobamos que persistió

En la misma página de Swagger: después del POST (“Try it out” → Execute), un **GET** del mismo recurso. Si volvió el JSON, persistió. Si fue 400, el GET no muestra esa fila. Sin phpMyAdmin.

# Qué pedía la entrega (y cómo lo mostramos en Swagger)

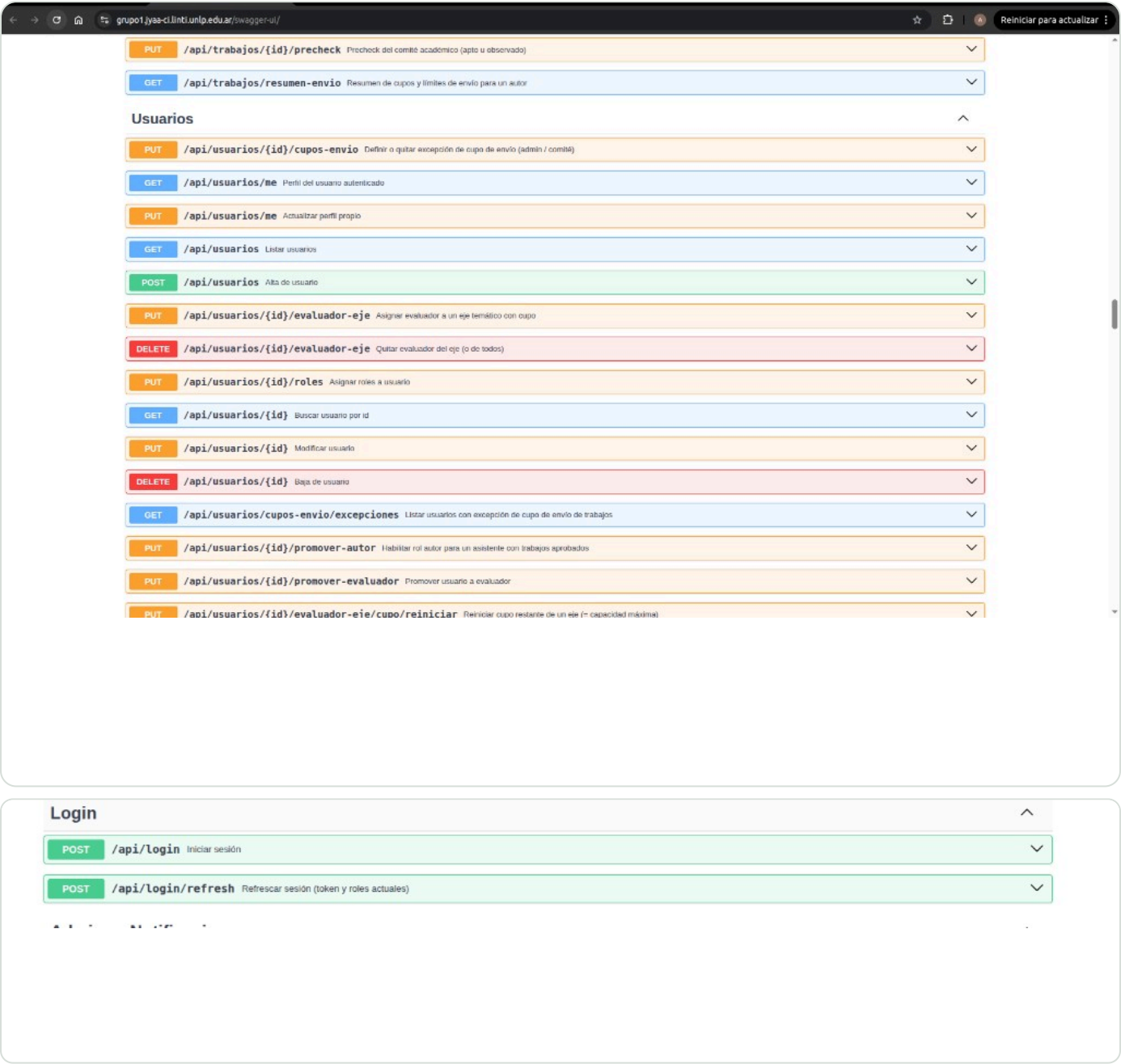
## Lo más importante que pedía la Entrega 4

- API REST con Jersey: endpoints por rol (Admin, OC, Evaluador, Autor, Participante).
- Capas limpias con CDI: Resource → Service → DAO, sin `new` a mano.
- Persistencia real en MySQL del curso (los datos sobreviven al reinicio).
- Swagger / OpenAPI para documentar y probar la API en vivo.
- DTOs en la respuesta (sin password ni ciclos de Jackson).
- Archivos (PDF) asociados a trabajos/pagos: se guarda la URL, no el binario en la entidad.

## Qué se ve en las capturas

La UI real en `/swagger-ui/` de nuestro servidor: el tag **Usuarios** con GET/POST/PUT/DELETE y sus `@Operation` (“Alta de usuario”, “Listar usuarios”...), y el tag **Login** con “Iniciar sesión” y “Refrescar sesión”. Sale solo de las anotaciones: nadie dibujó esa pantalla a mano.

Capturas reales de `grupo1.jyaa-ci.linti.unlp.edu.ar/swagger-ui/`: tags + operaciones documentadas; se prueba con Try it out y se verifica con GET.



# Angular habla con Java · por qué el navegador bloquea

SPA · CORS · orígenes

La E4 dejó la API REST (Java · Tomcat · Resource → Service → DAO · WAR; curl / Swagger). La E5 suma una **SPA Angular** que pide `/api/...` (reemplaza el prototipo React + `localStorage` de la E1). Pregunta: ¿cómo habla el navegador con el Java en otro servidor?

CORS es regla del **navegador** (no de Angular ni de Java). Un **origen** = protocolo + dominio + puerto.

## Orígenes distintos

A: `localhost:4200` · B: `grupo1...`

Para que A lea la respuesta de B, el servidor debe autorizar (`Access-Control-Allow-Origin`). Si no → “blocked by CORS policy”, aunque B haya respondido bien.

## Mismo origen

Página `grupo1.../` + API `.../api` = mismo origen (la ruta no cuenta). CORS no aplica — meta de producción.

## Hilo E5

① cómo conectar en local y en producción · ② un clic hasta los datos · ③ qué nos frenó.

# Tres formas: proxy · CORS real · producción

npm start · start: cors · WAR grupo1

## Forma 1 — proxy

npm start . Navegador ↔ solo localhost:4200 . Angular pide /api/... (environment.ts) . proxy.conf.json reenvía a grupo1: el salto lo hace el servidor de Angular. Día a día; no prueba cruce real.

## Forma 2 — CORS real

npm run start: cors . Sin proxy: URL completa de grupo1 (environment.cors.ts) . El navegador exige permiso. Sirve para verificar que el backend autoriza el cruce.

## Forma 3 — producción

La del usuario en grupo1 (no localhost) . Push → Docker compila Angular (/api) , lo mete en el backend, Maven → un WAR. Página + API = mismo origen → **sin CORS**.

## Forma 2 · permiso en el backend (casa CORS)

**Preflight:** un POST JSON (login) manda antes OPTIONS (“¿dejás que localhost:4200 ...?”) . Tres filtros Jersey (sin tocar Resources): CorsConfig (lista) · CorsRequestFilter (responde OPTIONS) · CorsResponseFilter (sella la respuesta) . Login: OPTIONS → sí → POST → Resource → sello → Angular.

## Qué buscaba la integración

Cliente real contra la API real · entender por qué “anda” en Swagger y a veces no en Angular · un push actualiza grupo1 sin npm en cada PC. La API de la E4 queda usable como interfaz de congreso.

# Del clic a la tabla · tres frenos

HttpClient · capas · CORS / push / WAR

## Historia: listar usuarios

La pantalla pide a `UsuarioService` → **HttpClient**. `listar()` devuelve un **Observable**; al suscribirse sale el GET. Camino: clic → Tomcat/Jersey → Resource → Service → DAO → MySQL → JSON → tabla. Entre las 3 formas solo cambia cómo llega el HTTP.

## Conceptos que entran (sin otra slide)

**Angular:** DI ( `@Injectable` , como CDI) · router (SPA) · guards ( `authGuard` / `roleGuard` ; JWT firme → E6) · HttpClient/RxJS.

**Java:** Jersey ( `/api/...` → método) · `web.xml` : `/api/*` → Jersey, resto → Angular (“truco del 404”). Filtros CORS = slide anterior.

## ① Swagger vs Angular

Swagger = mismo origen; `localhost` = cruce.  
Resolución: las tres formas.

## ② Push obligatorio

grupo1 solo corre el pipeline. Sin push, `start:cors` falla aunque el `.java` local esté bien.

## ③ WAR en Docker

La unión SPA + backend no es a mano: build multi-etapa (compilar → copiar → Maven). Detalle CI → Bloque 3.

**Cierre E5:** el navegador es el cliente oficial de la API. E6: producto completo en grupo1.

# Ya no es “integrar”: el congreso de punta a punta

por rol · configurable en grupo1

De “la API se consume desde el navegador” a “el congreso se gestiona completo en grupo1, por rol, sin redeploy por cada edición”.

**Veníamos (E5)**

SPA ↔ API · tres formas · clic hasta MySQL.

**Cierra la E6**

Inscripción → pago → trabajos → programa → cierre / certificados + JWT + más API + UI configurable.

**Admin / organización**

Datos · jornada · ventanas · cupos · aranceles (QR) · pagos / recibo / arqueo · aulas + mapa Leaflet · circulares · cierre y certificados.

**Comité académico**

Catálogos · plazos · cupos de evaluador · excepciones · precheck / asignación · tercer evaluador si empate 1–1.

**Autores · evaluadores · participantes**

Inscripción y pago · envío PDF / coautores · dictámenes · agenda · certificado. Avisos in-app y email.

**Público (sin sesión)**

Home · historia · programa + mapa · circulares. Sin JWT. (Demo en vivo → Bloque 4.)

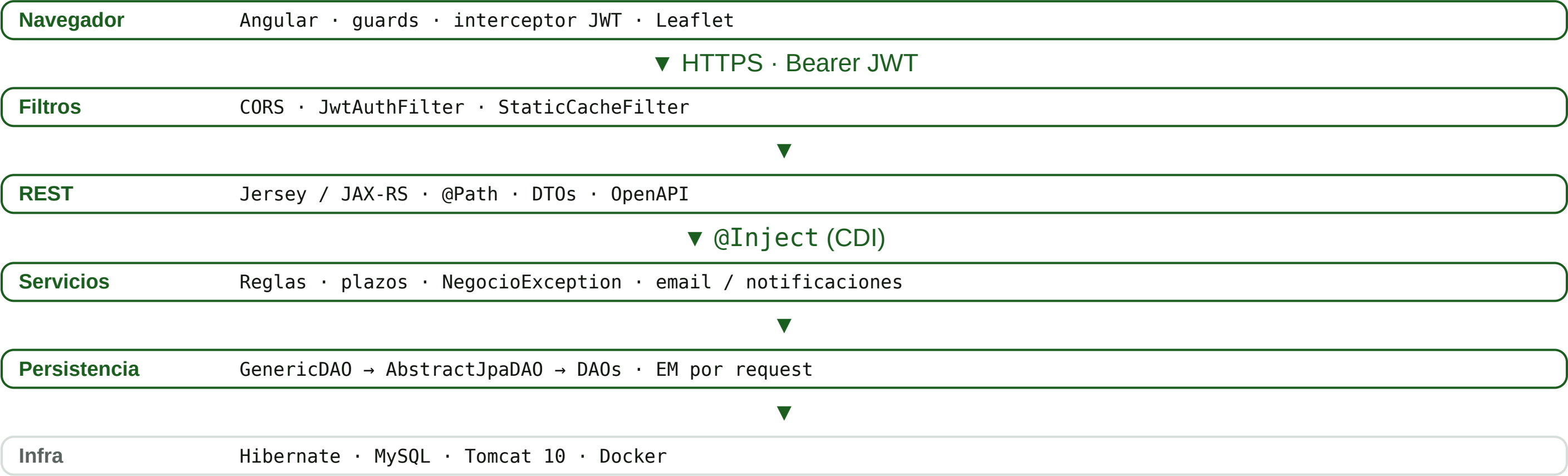
# JWT · WAR verificado · capas limpias

pase firmado · pipeline · un origen

**JWT — el hueco que cerró la E6**  
E5: login con usuario en `sessionStorage`; la API seguía abierta (Swagger / curl). E6: al login el servidor firma un JWT (email + roles); un **interceptor** lo manda en cada pedido ( `Authorization: Bearer ...` ). `JwtAuthFilter` valida en casi todos los `/api/...` (libres: login, registro, health, OpenAPI, lecturas públicas) y relee roles / cuenta activa en MySQL. Guards + 30 min de inactividad. API ampliada al ciclo del congreso; etapas / catálogos / aranceles / cupos / plazos desde la UI, sin redeploy.

**WAR — lo nuevo (sin recontar el Dockerfile)**  
A veces Docker reutilizaba caché → Angular viejo en grupo1. Fix: build limpio ( `--no-cache` ) + chequeo: si el WAR no trae `main-*.js` / estilos, la imagen no se publica.  
**Cierre E6:** congreso de punta a punta en [grupo1...](#).

Un pedido baja por capas (E3 mezclaba; E4 separó con CDI; E5/E6 sumaron Angular, CORS/JWT y el WAR):





# 03

## Cómo trabajamos

Organización del grupo y las herramientas que la sostuvieron.



# Por flujo del congreso · Git, Docker y CI

3 integrantes · un push → grupo1

Primero backend los tres juntos; después Angular. Para no pisarnos: cada uno dueño de un **flujo completo** (entidad → pantalla).

### Benjamín

Inscripción, aranceles, pagos, recibos, arqueo, facturas · usuarios · gran parte del modelo y la API.

### Alcides

Trabajos, comité, evaluadores, dictámenes · programa, aulas, cronogramas y mapa.

### Lucas

Login, JWT, roles · certificados, circulares, notificaciones · pipeline, Docker y pruebas.

### Cómo versionamos y publicamos

- GitHub día a día · GitLab cátedra para entregar y desplegar
- Rama por entrega + ramas chicas · `main` solo si el pipeline está verde
- CI (`jyaa`): arma imagen → frena/limpia contenedor → publica · Traefik HTTPS
- El WAR se verifica antes de publicar (Angular adentro)

### Pipeline en 4 pasos

**1 Armar** (sin caché vieja) · **2 Frenar** · **3 Limpiar** · **4 Publicar** en `grupo1.jyaa-ci.linti.unlp.edu.ar`

Dueño de un flujo = menos choques al merge. Piezas compartidas (usuario, config): se hablan primero.

`Git` `GitLab CI` `Docker` `Traefik` `Maven`

# 04

## Demo y reflexión

Capturas: configurar + programa + fallidos ... En vivo: registro → Autor y programa público

# Configurable: cuándo inicia y cierra el congreso

sin esto el usuario no puede usar el sistema

Antes de que alguien se registre: el admin define **ventanas** y **plazos** (inscripción, envío, evaluación). Si no están abiertos, no hay “Nuevo trabajo”.

### Ventanas de tiempo del congreso

Cada bloque se guarda y notifica por separado. El congreso dura **3 días corridos** (día 1 = inicio, día 3 = fin automático).

#### 1. Inicio y fin del congreso

Notifica a todos. Al guardar, el programa (día 1/2/3) se reacomoda a las nuevas fechas.

Inicio del congreso (día 1)

24/07/2026

Fin del congreso (día 3, automático)

26/07/2026

Motivo del cambio

Ej. Postergación por causas climáticas

Guardar fechas del congreso

#### 2. Período de inscripción

Notifica solo por este cambio (a todos).

Inscripciones desde

11/07/2026

Inscripciones hasta

21/07/2026

Motivo del cambio

Ej. Se amplía el plazo de inscripción

Guardar período de inscripción

### 3. Plazo de envío de trabajos

Notifica a asistentes y autores.

Envío de trabajos hasta

24/07/2026

Motivo del cambio

Ej. Se extiende el plazo de envío

Guardar plazo de envío

### 4. Plazo de evaluación

Notifica solo a evaluadores.

Evaluación hasta

24/07/2026

Motivo del cambio

Ej. Se extiende el plazo de evaluación

Guardar plazo de evaluación

El comité también puede editar solo la fecha límite de envío de trabajos desde su panel.

Ventanas — inicio/fin del congreso e inscripción

Plazos — envío de trabajos y evaluación

Captura · config → aranceles

35 / 47

# Configurable: aranceles y datos de pago

sin precios publicados no hay inscripción clara

El admin publica **precios por categoría** (socio, estudiante, extranjero...) y **cómo pagar** (alias/CBU, instrucciones, QR). Con eso el participante sabe cuánto y a dónde transferir.

Estado

Aranceles publicados: los participantes ya ven precios y datos de pago.

Precios por categoría

Moneda ARS o USD (extranjero suele ser USD).

|                                       |              |            |
|---------------------------------------|--------------|------------|
| Socio/a SAAE                          | Monto 75500  | Moneda ARS |
| No socio/a                            | Monto 154997 | Moneda ARS |
| Estudiante de grado                   | Monto 37500  | Moneda ARS |
| Productor/a de organización/comunidad | Monto 50000  | Moneda ARS |
| Investigador/a                        | Monto 150000 | Moneda ARS |
| Extensionista                         | Monto 150000 | Moneda ARS |
| Docente                               | Monto 150000 | Moneda ARS |
| Extranjero/a                          | Monto 170    | Moneda USD |

Datos de pago

Alias / CBU / CVU leandro.lp287

Instrucciones (opcional) titular: leandro paredes  
banco: provincia

QR de pago

Subir imagen QR (PNG/JPG) | Seleccionar archivo Sin archivos seleccionados

Aranceles — categorías · moneda · alias/CBU · instrucciones

# Cómo habilitamos el programa (datos y franjas)

configurable · admin

Parte de lo mismo: armar el congreso sin redeploy. Pasos 1 y 2 de la guía.

Guía · Crear programa — paso 1 de 4

1 Datos

2 Franjas

3 Aulas

4 Actividades

— Hub Congreso

Siguiente: Franjas →

Cuando creás actividades, elegís aula y franja; se reflejan en Programa.

Datos del congreso

Nombre, edición y sede (aparecen en cabecera, inicio y certificados).

Nombre

Congreso Argentino de Agroecología

Edición

V

Sede

La Plata

Guardar datos

Ubicación en el mapa

Definí el punto de la sede. Al verla se aplica un rango alrededor; las aulas solo pueden ubicarse dentro de ese rango.

Ver ubicación del congreso en el mapa

Editar ubicación en el mapa

Centro guardado: -34.91228, -57.93091

1 · Datos — nombre, edición, sede, mapa

— Volver a Congreso

Guía · Crear programa — paso 2 de 4

1 Datos

2 Franjas

3 Aulas

4 Actividades

— Datos

Siguiente: Aulas →

Cuando creás actividades, elegís aula y franja; se reflejan en Programa.

1. Horario de actividades (jornada)

Primero el rango en el que puede haber actividades. Podés dejar un horario común o personalizar cada día.

Inicio (común)

09:00 a.m.

Fin (común)

08:00 p.m.

Personalizar jornada por día

Día 1

Inicio

08:00 a.m.

Fin

09:00 p.m.

Usar común

Día 2

Inicio

--:-- --:--

Fin

--:-- --:--

Usar común

Día 3

Inicio

--:-- --:--

Fin

--:-- --:--

Usar común

Guardar jornada

2 · Franjas — jornada / horarios

Captura · programa 1/2

37 / 47

# Aulas y actividades (luego lo vemos en vivo)

mapa Leaflet · mesas / pósters / talleres

[← Volver a Congreso](#)

Guía · Crear programa — paso 3 de 4

1 Datos

2 Franjas

3 Aulas

4 Actividades

← Franjas

Siguiente: Actividades →

Quando creás actividades, elegís aula y franja; se reflejan en Programa.

[Crear nueva aula](#)

Listado de aulas

Al programar actividades se elige un aula; se controlan choques de horario. La ubicación en el mapa se carga al crear o editar el aula.

| NOMBRE     | CAPACIDAD | UBICACIÓN                          | MAPA                                     | ESTADO | ACCIONES                                             |
|------------|-----------|------------------------------------|------------------------------------------|--------|------------------------------------------------------|
| aula 1     | 99        | Facultad de Agronomía - Edificio A | <a href="#">Ver ubicación en el mapa</a> | Activa | <a href="#">Editar</a><br><a href="#">Desactivar</a> |
| Aula 2     | 1         | Facultad de Agronomía - Edificio A | <a href="#">Ver ubicación en el mapa</a> | Activa | <a href="#">Editar</a><br><a href="#">Desactivar</a> |
| Aula magna | 150       | FCA edificio central               | <a href="#">Ver ubicación en el mapa</a> | Activa | <a href="#">Editar</a><br><a href="#">Desactivar</a> |

V V Congreso Argentino de Agroecología — La Plata 2026 · Angular + REST Jersey

3 · Aulas — capacidad y ubicación en mapa

[← Volver a Congreso](#)

Guía · Crear programa — paso 4 de 4

1 Datos

2 Franjas

3 Aulas

4 Actividades

← Aulas

Ver programa →

Quando creás actividades, elegís aula y franja; se reflejan en Programa.

Crear mesa temática

Presentaciones orales agrupadas por eje

Crear mesa redonda

Debate con panelistas invitados

Crear sesión de pósters

Bloque de pósters en el cronograma

Crear taller

Taller oficial del programa

Crear conferencia

Conferencia magistral u otro bloque oratorio

V V Congreso Argentino de Agroecología — La Plata 2026 · Angular + REST Jersey

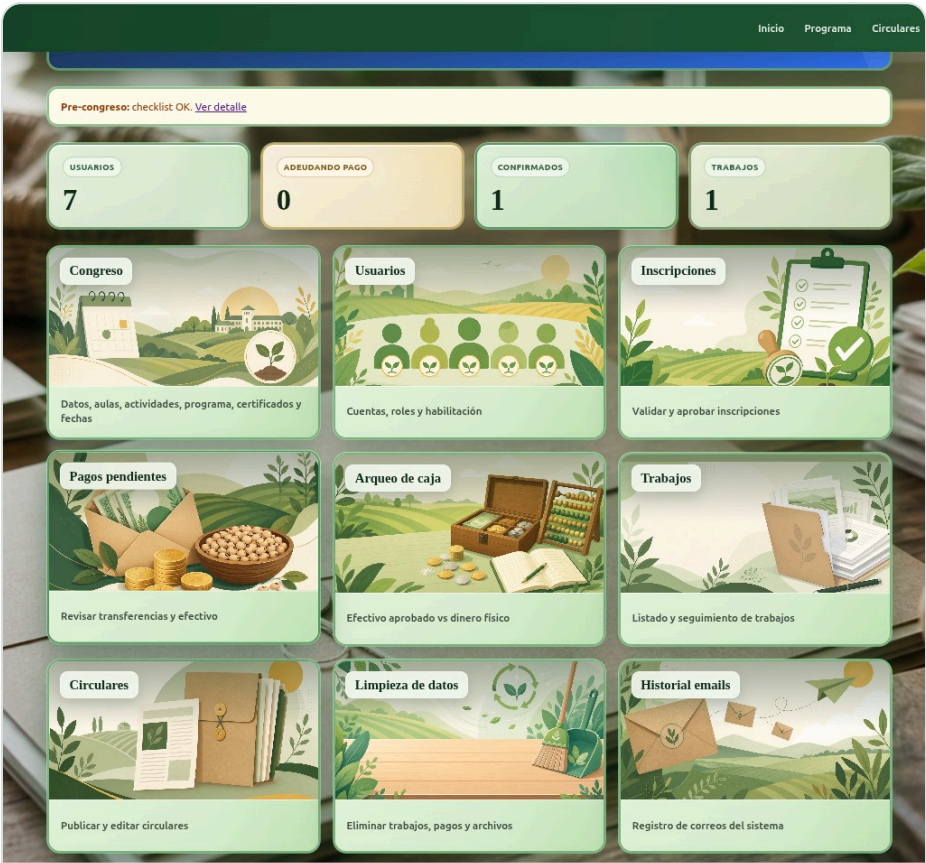
4 · Actividades — mesas, pósters, talleres...



# Tres paneles: admin · comité · evaluador

adelanto de roles (además: asistente / autor)

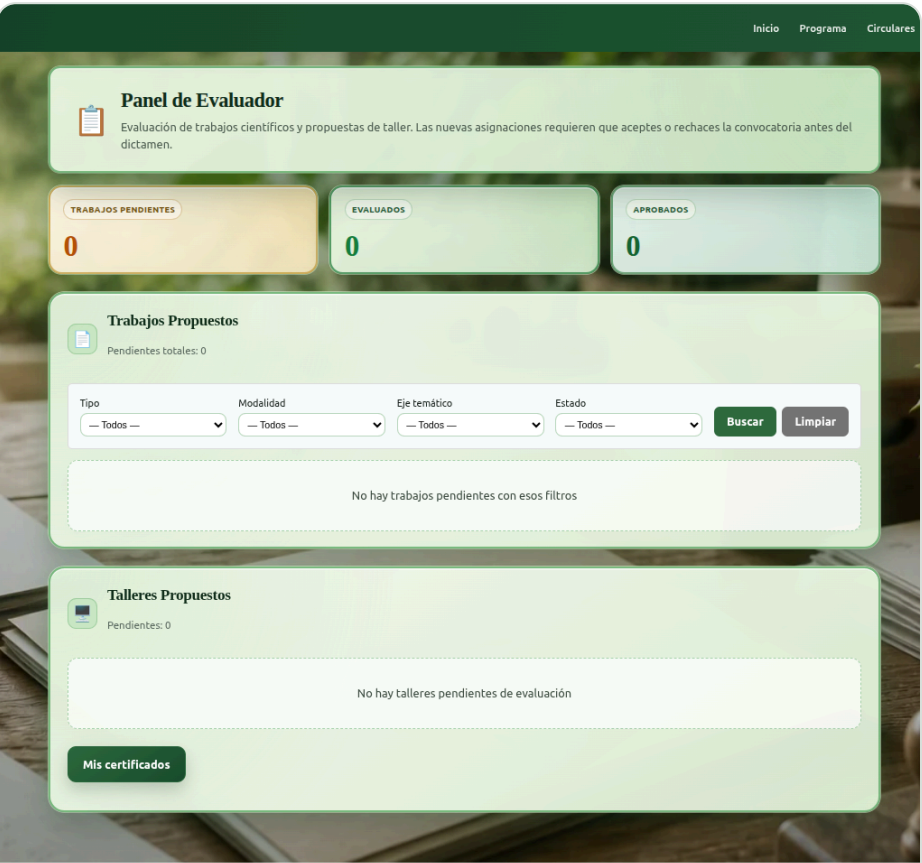
Configurado el congreso: un usuario opera desde paneles distintos según el rol. Acá los tres de organización/evaluación; en vivo entran también **asistente** y **autor**.



Admin — congreso, pagos, usuarios...



Comité — precheck, ejes, cupos



Evaluador — dictámenes asignados

**A1** Precheck NO — observar

**A2** Estado observado — aún no hay rol Autor



# En ambos caminos: el asistente puede reenviar

A3 y B3

Corrige desde el panel asistente y vuelve a mandar (sirve para precheck NO y para evaluación negativa).

La decisión final la toma la comisión académica.

Tipo de envío

Trabajo científico

Coautores

El autor/a de la cuenta ya figura como responsable. Agregá coautores/as si corresponde (ilimitados).

Alexis

Quitar

Agregar autor

Archivo PDF (obligatorio para enviar)

Seleccionar archivo

Sin archivos seleccionados

Ya hay un PDF cargado — [ver actual](#)

Archivo Word .docx (opcional, p. ej. para control de cambios)

Seleccionar archivo

Sin archivos seleccionados

Ya hay un Word cargado — [descargar actual](#)

Reenviar trabajo

Mis trabajos (rol asistente)

Suelos salinos en tierra 1

Planificación y desarrollo territorial • Oral • Trabajo científico • Precheck 1/3 • Revisión 0/2

Coautores: Alexis

[PDF](#) [Word \(.docx\)](#)

Observado en prevalidación (envío como asistente). Corregí y reenviá desde acá. Si el comité aprueba el trabajo, el rol Autor se habilita automáticamente.

Editar y reenviar

Enviado como Asistente

Observado en prevalidación

[← Panel asistente](#)

[illegible]

Fallido B1  $\rightarrow$  B2

# Los dos piden correcciones → reenvío

evaluación negativa

Tras asignar evaluadores al trabajo: ambos piden modificaciones profundas. El asistente **reenvía** (misma pantalla que vimos).

Trabajos1 total

Título

Buscar por título

Eje temático

Buscar eje

Estado

— Todos —

Buscar

Limpiar

Suelos salinos en tierra 1

Requiere correcciones (evaluación)

Planificación y desarrollo territorial

TRABAJO\_CIENTIFICO • Modalidad: Oral • Enviado como: Asistente

Categoría autor/a: Sin categoría

Asignaciones: 2 • Reviews: 2 (✓ 0 / ✗ 2)

Precheck 1/3

Revisión 1/2

Página 1 de 1 — 1 registro(s)

Anterior

Siguiente

Suelos salinos en tierra 1

Requiere correcciones (evaluación)

Estudio de caso en la región pampeana

Participante: gonzales roberto Enviado como Asistente

Este trabajo se presentó con cupo de asistente. Al aprobar el dictamen final, el rol Autor se habilita automáticamente.

Los evaluadores rechazaron este trabajo (1/2 reenvíos). Fue enviado como asistente: el participante debe corregirlo y reenviarlo desde el panel asistente. Tras dos evaluaciones favorables y el dictamen final del comité, el rol Autor se habilita automáticamente.

Evaluaciones registradas

benjamin, mantilla

rodriguezmantilla123@gmail.com

Asignación aceptada

Decisión

Precisa modificaciones profundas y volver a revisar

Modalidad sugerida

Oral

Comentario a autorxs

falta corregir todo al usuario

Comentario interno (comisión)

te faltan corregir estos puntos

Archivo de correcciones

10.55.56\_Cielorrasos-P1-PT-A3.pdf

Ver rúbrica (criterios)

budnik, lucas

lucasbudnik@hotmail.com.ar

Asignación aceptada

Decisión

Precisa modificaciones profundas y volver a revisar

Modalidad sugerida

Oral

Comentario a autorxs

necesita corregir

Comentario interno (comisión)

corregi en todo esto

Archivo de correcciones

Entrega 5 -guia.pdf

Ver rúbrica (criterios)

B2 Requiere correcciones (evaluación)

# En vivo: del registro al rol Autor

~5 min · grupo1

Lo fallido quedó en captura. Acá el camino completo que **sí** termina bien.

Producción: <https://grupo1.jyaa-ci.linti.unlp.edu.ar/>

## En el navegador (orden)

1. **Registro + inscripción + pago** (comprobante).
2. **Admin valida el pago** → queda **asistente**.
3. Asistente **envía** el trabajo.
4. Comité: **precheck OK** + asigna **2 evaluadores**.
5. Los dos dictaminan **OK**.
6. Comité: **dictamen final** aprueba.
7. Se habilita el rol **Autor**.

# El programa público ya cargado

~1 min · sin login

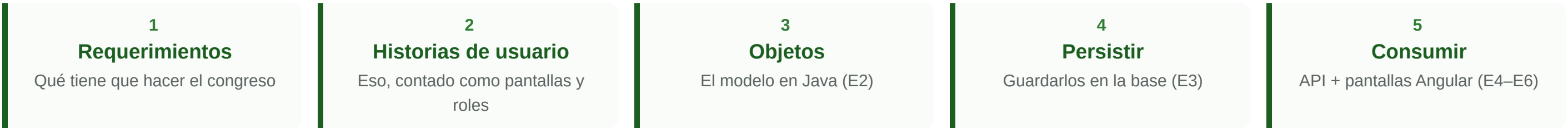
Resultado de la configuración: **programa** con días, actividades y **mapa de aulas** (Leaflet), sin JWT.

Qué señalar: home / programa · pinchos · lo configurable que vimos en captura ya se ve publicado. Cierre demo: punta a punta en grupo1.

# Qué nos llevamos

1 min

No fue “aprender una herramienta suelta”: fue **armar una aplicación de punta a punta**, en el mismo orden en que se construye un sistema real.



Y en paralelo

Desde temprano usamos **GitLab, Docker y el pipeline** para publicar en grupo1. Así cada avance no quedó en la notebook de alguien: quedó en el servidor, visible para todos.

Si hubiera una entrega más: contraseñas hasheadas, permisos por rol en el servidor, validar datos en la API y tests automáticos en el pipeline.

# Gracias. ¿Preguntas?

---

Sistema de gestión del V Congreso Argentino de Agroecología

**Aplicación:** grupo1.jyaa-ci.linti.unlp.edu.ar    **API:** /api/health · **OpenAPI:** /api/openapi.json

**Repositorio:** GitLab de la cátedra — grupo 1